

Stealth In, Linked Out:

The Consolidation Gap of a Deployed Stealth-Address Wallet
and What It Costs to Close It on a 2018 Phone

[Author name]

EnergíaSonora / ChatWallet

xunorus@gmail.com

Problem

Stealth addresses (ERC-5564 [1]) make *receiving* private: every payment lands on a fresh address that only the recipient can link to themselves. *Spending* is a different story. Ethereum has no multi-input transaction, so paying an amount larger than any single stealth balance requires N transfers from N stealth addresses to the same destination within seconds, a pattern that re-links the very identities the scheme was designed to separate. We call this the *consolidation gap*. We report on it from a deployed wallet, measure what it costs to close it with today's shielded pools on a commodity 2018 phone, and show that the pools' anonymity sets are fragmented across chains in a way that leaves low-fee users unprotected.

System: StealthPay in ChatWallet

ChatWallet is an open-source, self-custodial wallet with end-to-end encrypted chat over XMTP, oriented to stablecoin payments on Base, distributed as a PWA and an Android app. It implements StealthPay v1, a public specification with reference code and test vectors [3]:

- keys derived deterministically from the wallet seed;
- ERC-5564 scheme 1 (secp256k1 with view tags) against the canonical Announcer;
- a chain scanner that filters announcements by view tag before the costly point addition and recovers all payments from the seed alone;
- an optional in-chat notice that speeds up detection but never replaces the on-chain announcement.

Spending is where the engineering lives:

- **Coin selection.** Stealth balances are treated as non-fungible coins, never as one balance. The wallet always prefers the *smallest single coin that covers the payment*, merges only when unavoidable, and tells the user how many identities will be joined before confirming.
- **Gas without linkage.** A stealth address holding USDC has no ETH, and funding it from the main wallet links the two. Instead the wallet signs two ERC-3009 [2] `transferWithAuthorizations` (payment and fee), which a relayer submits atomically. The relayer's ledger records amounts and fees but no addresses or transaction hashes, so it does not become the linkage database the scheme exists to avoid.

Where linkage still leaks

Running the system exposed four residual channels:

1. **Consolidation:** N same-destination transfers in the same minute. Coin selection mitigates this but cannot remove it.
2. **Gas funding:** closed by ERC-3009 plus the relayer, at the cost of the relayer seeing the (stealth address, destination) pair in transit.
3. **Destination:** paying the user's own main address, or a counterparty who knows them.
4. **Read metadata:** the RPC that polls all of a user's stealth balances can cluster them.

Channel 1 cannot be fixed inside a transparent ledger. It needs a shielded pool in which coins are merged privately and only the final payment exits.

Closing the gap: three deployed pools, measured

We compare the three pools a wallet could integrate today, using L2BEAT’s privacy dashboard [4] (as of 4 Oct 2026) and our own measurements.

	zk.money (Aztec) [5]	Privacy Pools [6]	Railgun [7]
Private spending inside	yes	no (deposit/withdraw)	yes
Chains	Ethereum portal	Ethereum	Ethereum, Base
TVL	\$17.3k	\$9.43M	\$112M (Ethereum)
Anonymity set	launch-stage	6–83 depositors/asset	large on Ethereum
Exit depends on	live AWS Nitro enclave	ASP inclusion (else public ragequit)	DAO upgrades (7-day window)
Per-tx cap	<2,583 DAI	per-asset limits	none
Proof on 2018 phone	≈1 min (vendor docs, unspecified device)	6.0 s (ours)	11.5–41 s + 22.5 s POI (ours)

Measurement method. Device: Samsung Galaxy S9 (Exynos 9810, 2018, Android 10), Chrome 154, snarkjs 0.7.6 WASM in the browser. This is the path a PWA or WebView wallet uses; native provers were not measured. Circuit artifacts were fetched from the IPFS root used by the official Railgun SDK 11.2.0 and checked against the SHA-256 table shipped in the SDK. For Railgun’s JoinSplit we built *valid* witnesses by hand from the public circuit [8]:

- an EdDSA-Poseidon signature over the public-signal hash;
- nullifiers $\text{Poseidon}(nk, i)$;
- note commitments with a depth-16 Merkle membership proof;
- value balance between inputs and outputs, with USDC on Base as the token.

Every proof verified against the official verification key. The deployed Proof-of-Innocence (POI) circuit differs from its public repository, so we timed `groth16.prove` on a synthetic witness of the correct length. Calibrated against real JoinSplit proofs on the same device, synthetic timings were within 3% (11.3 vs. 11.5 s; 40.1 vs. 41.0 s).

Circuit (inputs×outputs)	Use	Artifacts (compressed)	S9 (median of 3)	Laptop (M-series)
JoinSplit 1×2	pay from one note	4.4 MB	11.5 s (13.0 cold)	1.3 s
JoinSplit 2×2	merge two	5.7 MB	14.7 s	–
JoinSplit 3×2	merge three stealth coins	7.9 MB	20.6 s	–
JoinSplit 8×2	consolidate eight	15.9 MB	41.1 s	4.4 s
POI 3×3 (synthetic witness)	innocence proof	8.9 MB	22.5 s	2.5 s
Privacy Pools withdraw [6]	exit	19.5 MB	6.0 s	2.3 s

A private payment that merges three stealth coins therefore costs about **43 s of proving** on a 2018 phone (JoinSplit 3×2 plus POI), plus about 17 MB of artifacts on first use. On a laptop it costs about 4 s. The phone-to-laptop ratio is a stable 9–10× across circuits. Proving is feasible but far from instant, and this latency is precisely the step users would be asked to wait through for privacy.

Finding: anonymity sets do not follow users across chains

Our target users hold small amounts on Base, where fees are low, so the natural design is to keep the whole stealth → shield → spend flow on Base. Railgun has been deployed on Base since 20 Aug 2026 (proxy 0x0047d1f9..., block 50,224,711). We enumerated every event and token transfer on that proxy through 6 Oct 2026 and found:

- **one Shield** ever, of 0.25 USDC on 8 Sep (0.249375 net of the 0.25% fee);
- **zero Transact** and **zero Unshield** events.

The SDK’s own network configuration notes that Base cannot yet be loaded by the wallet SDK. The \$112M crowd lives on Ethereum mainnet, where fees are far higher for small payments, and bridging from Base to reach it is itself a public, linkable hop. A user who shields on Base today is the pool’s second depositor. The other options show the same tension in different forms:

- **zk.money** [5] has a polished payments UX (human-readable tags, payment links, XMTP requests), but at launch it held \$17.3k across 762 deposits.

- **Privacy Pools** reports 6–83 unique depositors per asset in recent windows.

Deployment is not anonymity: a wallet integrator must check where the crowd actually is, chain by chain, before promising privacy.

Naming and the privileged resolver

Human-readable payment names interact with all of this. zk.money’s ENS wildcard resolver returns a fresh deposit address per lookup, but the operator can re-derive every such address (L2BEAT: “privileged insider: link exposed”). We face the same design choice for ChatWallet’s `@name` links. An extended-public-key resolver gives fresh, seed-recoverable addresses without on-chain announcements, but its operator learns the same links. We will discuss how this compares with keeping ERC-5564 meta-addresses, which the resolver never sees, for wallet-to-wallet payments.

Takeaways for the workshop

1. Stealth receiving is a solved, deployable problem. Stealth *spending* is not, and coin selection plus gasless relaying only shrink the leak.
2. Closing it with a shielded pool costs roughly 30–45 s of proving per payment on a 2018 phone in WASM. Native provers and artifact bundling are the next engineering steps; we will report them if available by the workshop.
3. The binding constraint is not cryptography but *where the anonymity set lives*: the cheap chain has the users, the expensive chain has the crowd.
4. The trust bases differ in kind: a TEE that must stay alive, an association-set provider, or token governance. A wallet has to choose which one to expose its users to.

All code, inputs, raw results and scripts are public and reproducible: <https://github.com/energiasonora/chatWallet/tree/ius-vinculo-g1/investigacion/wppt2026>.

Publication status. Unpublished work in progress, based on a deployed system.

Use of AI tools. In line with the Asiacrypt 2026 policy, the authors disclose that a generative AI assistant (Claude, Anthropic) was used substantially to draft this abstract and to write the measurement scripts. The authors reviewed, verified and take full responsibility for all content and data.

References

- [1] T. Wahrstätter et al. ERC-5564: Stealth Addresses. Ethereum Improvement Proposals, 2022.
- [2] ERC-3009: Transfer With Authorization. Ethereum Improvement Proposals, 2020.
- [3] StealthPay v1 specification, reference implementation and vectors. <https://github.com/energiasonora/stealthpay>
- [4] L2BEAT Privacy dashboard. <https://l2beat.com/privacy>, accessed 4 Oct 2026.
- [5] Aztec Labs. zk.money documentation. <https://docs.zk.money>, accessed 4 Oct 2026.
- [6] V. Buterin, J. Illum, M. Nadler, F. Schär, A. Soleimani. Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium. 2023.
- [7] Railgun. <https://railgun.org>; SDK @railgun-community/wallet 11.2.0.
- [8] Railgun circuits v2. <https://github.com/Railgun-Privacy/circuits-v2>